OpenMP — Lesson 8

False sharing

Gustavo Lozano

November 2026

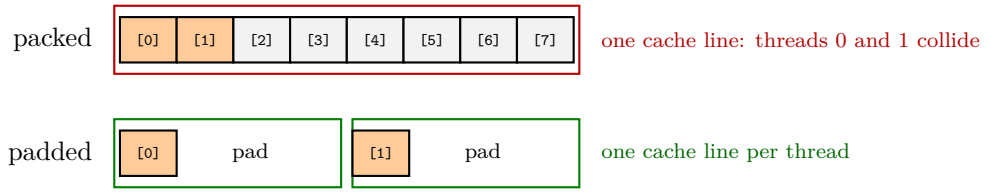
Source: Chapman, Jost & van der Pas [1], Ch. 5 "How to Get Good Performance" (false sharing).

1 The idea

Lessons 2–7 were about *correctness*. This lesson is about *speed*, and it shows a problem that the source code does not reveal.

A CPU core does not read single bytes from memory. It copies a whole **cache line**, usually 64 bytes, into its private cache. When a core *writes* to a line, the hardware must invalidate every other core's copy of that line, so that no core keeps a stale value. This is called *cache coherence*.

Now let each thread increment its own counter, `packed[id]`. There is **no data race**: no two threads touch the same variable. But 8 `long`s fit in one 64-byte line, so all the counters live in the *same* line. Every write by thread 0 invalidates the line in thread 1's cache and vice versa. The line bounces between the cores on every increment. This is **false sharing**: the threads share a cache line, not data.



2 Two fixes

Fix	How
Padding	make each thread's element occupy a whole cache line, e.g. with C11 `_Alignas(64)`, so different threads never write the same line
Local variable	accumulate in a variable private to the thread (it lives in a register or on that thread's stack) and write the shared array *once* at the end. This is what `reduction` does for you (Lesson 3)

3 The code

The same work three times: each thread does 10^8 increments of its own counter.

`false_sharing.c`

```c
/*
 * false_sharing.c -- OpenMP Lesson 8: false sharing.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 5 (How to Get Good Performance: false sharing).
 *
 * Compile (no -O flag, so every increment really goes to memory):
```

```c
 *     gcc -Wall -fopenmp -o false_sharing false_sharing.c
 * Run (use as many threads as you have cores, see `nproc`):
 *     OMP_NUM_THREADS=2 ./false_sharing
 *
 * Check your cache line size with:   getconf LEVEL1_DCACHE_LINESIZE
 */

#include <stdio.h>
#include <omp.h>

#define ITERS      100000000L  /* increments done by EACH thread        */
#define MAXT       64          /* max threads                           */
#define CACHE_LINE 64          /* bytes per cache line on most x86 CPUs */

/* (1) PACKED: counters next to each other. 8 longs fit in ONE 64-byte
       cache line, so up to 8 threads write into the same line.         */
long packed[MAXT];

/* (2) PADDED: _Alignas (C11) puts each counter at the start of its own
       64-byte block, so sizeof(padded_t) == 64 and every thread writes
       to a DIFFERENT cache line.                                        */
typedef struct {
    _Alignas(CACHE_LINE) long v;
} padded_t;
padded_t padded[MAXT];

int main(void)
{
    int p = omp_get_max_threads();
    double t;

    printf("threads = %d, sizeof(padded_t) = %zu bytes\n\n", p, sizeof(padded_t));

    /* ---- (1) packed: no data race (each thread has its own slot),
       but the slots share a cache line -> FALSE SHARING.               */
    t = omp_get_wtime();
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        for (long i = 0; i < ITERS; i++)
            packed[id]++;
    }
    printf("packed : %.3f s\n", omp_get_wtime() - t);

    /* ---- (2) padded: same work, one cache line per thread. ---- */
    t = omp_get_wtime();
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        for (long i = 0; i < ITERS; i++)
            padded[id].v++;
    }
    printf("padded : %.3f s\n", omp_get_wtime() - t);

    /* ---- (3) local: accumulate in a private variable and write the
       shared array only ONCE at the end. Usually the best fix.         */
    t = omp_get_wtime();
    #pragma omp parallel
    {
        int  id  = omp_get_thread_num();
        long sum = 0;                       /* private, lives in a register
                                               or this thread's stack     */
        for (long i = 0; i < ITERS; i++)
            sum++;
```

```
        packed[id] += sum;                  /* one write per thread */
    }
    printf("local  : %.3f s\n", omp_get_wtime() - t);

    return 0;
}
```

```
gcc -Wall -fopenmp -o false_sharing false_sharing.c
OMP_NUM_THREADS=2 ./false_sharing
```

Output on a 2-core machine (your times will differ):

```
threads = 2, sizeof(padded_t) = 64 bytes

packed : 0.423 s
padded : 0.200 s
local  : 0.042 s
```

Key ideas

- All three versions are *correct*: no thread ever touches another thread's counter. They differ only in memory layout.
- Each thread does the same work in every version, so with enough cores the time should not depend on the number of threads. `packed` breaks this rule: it gets slower as threads are added.
- False sharing is invisible in the source code. You find it by measuring, and by asking: do threads *write* to addresses closer than 64 bytes apart?
- Reading is not the problem. Only writes force the other copies to be invalidated.

4 The experiment

Experiment

Question: how does false sharing scale with the number of threads?

1. Check your cache line size and number of cores:

   ```
   getconf LEVEL1_DCACHE_LINESIZE
   nproc
   ```

2. Run with every thread count from 1 up to `nproc`:

   ```
   for t in $(seq 1 $(nproc)); do echo "== $t"; OMP_NUM_THREADS=$t ./false_sharing | tail
       -3; done
   ```

3. Write a table: rows = thread count, columns = `packed`, `padded`, `local`.

What to observe and answer.

1. With 1 thread, `packed` and `padded` take the same time. Why?
2. How does `packed` change as threads are added? And `padded`?
3. Why is `local` the fastest, even with 1 thread?

Answers (read after trying). (1) With one thread no other core holds a copy of the line, so nothing is invalidated. Both versions do the same instructions. (2) `packed` slows down: every extra thread writing into the same line adds more invalidations and transfers between cores (on the 2-core test machine it went from 0.20 s to 0.42 s). `padded` stays roughly constant, which is the ideal behaviour when each thread has a fixed amount of independent work. (3) Every iteration of `packed` and `padded` must compute an address in a global array and update it. `local` only updates a private variable that no other core ever touches, so it stays in this core's cache, and the shared array is written once. It avoids false sharing *and* the indexing work. Rule: accumulate privately, write shared data rarely.

Next lesson. Locks and `ordered`: protecting shared data with the lock routines of the OpenMP library, and forcing part of a parallel loop to run in sequential order (Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.